Sift.

AI Resume Screening System - By KEVAL SHAH

A hybrid resume-screening application that ranks a batch of candidates against a single job description using semantic embeddings, skill-keyword coverage, and fuzzy text overlap — with an optional LLM layer that explains each ranking in plain language.

Technical Documentation · Version 1.0
Stack: FastAPI · Sentence Transformers · scikit-learn · RapidFuzz · Vanilla JS
Prepared for: HR / Recruitment AI Evaluation

Contents

01 Overview & Business Problem

Recruiters routinely receive dozens to hundreds of resumes for a single opening. Reading each one and comparing it fairly against the job requirements is slow, inconsistent, and prone to keyword bias — a strong candidate who phrases their experience differently can be missed entirely.

Sift automates the first-pass screen. A recruiter pastes one job description, uploads a stack of resumes, and receives a ranked shortlist in seconds. Each candidate comes with a transparent score breakdown and a short written rationale, so the tool *assists* human judgement rather than replacing it.

What it solves

- **Speed:** ranks an entire batch in one pass instead of manual one-by-one review.
- **Consistency:** every candidate is scored on the same three axes with the same weights.
- **Beyond keywords:** semantic embeddings catch meaning, so "built retrieval systems" matches "information retrieval engineer" even without shared words.
- **Explainability:** each ranking shows *why*, including matched and missing skills.

02 How It Works — The Scoring Pipeline

Each resume is converted to text and scored against the job description on three independent axes. The three sub-scores are combined into a single weighted composite, scaled to 0–100.

Axis	Weight	What it measures	Technique
Semantic fit	55%	Conceptual match between resume and JD, independent of exact wording.	Sentence Transformer embeddings (`all-MiniLM-L6-v2`), cosine similarity.
Skill coverage	35%	How many required skills appear in the resume.	Substring + fuzzy match (RapidFuzz `partial_ratio ≥ 90`).
Text overlap	10%	Broad lexical overlap as a tiebreaker.	RapidFuzz `token_set_ratio` .

> **Composite formula:**
> `score = (0.55 × semantic) + (0.35 × skill_coverage) + (0.10 × fuzzy)` , then × 100.

Why a hybrid?

Pure keyword matching is brittle — it rewards resumes that happen to copy the JD's vocabulary. Pure semantic matching can over-reward fluent but irrelevant writing. Combining both, plus an explicit skills check, gives a score that is robust and easy to defend to a hiring manager.

The LLM rationale layer

For the top-N candidates, the app sends the score breakdown and a resume excerpt to Claude, which returns a 2–3 sentence rationale: strongest fit, biggest gap, and why they rank where they do. If no API key is configured, a deterministic fallback summary is generated instead, so the application always runs.

03 System Architecture

A single FastAPI process serves both the JSON API and the static frontend, so the whole app runs from one URL with no separate web server to configure.

```
Browser (index.html)
      |  multipart upload: JD + skills + resume files
      v
FastAPI  /api/screen
      |
      |--> parser.py     PDF / DOCX / TXT  -->  plain text
      |--> scorer.py     embeddings + skill match + fuzzy --> ranked list
      |--> explainer.py  top-N --> Claude rationale (or fallback)
      |
      v
JSON response  -->  rendered as ranked cards in the browser
```

Design choices

- **Stateless & in-memory:** nothing is persisted; each request is self-contained. Privacy-friendly and simple to deploy.
- **Model cached once:** the embedding model loads a single time per process (LRU-cached), not per request.
- **Graceful degradation:** unreadable files are skipped and reported, not fatal; the LLM layer falls back cleanly without a key.

04 Project Structure

```
resume-screener/
├── backend/
│   ├── main.py               # FastAPI app + routes; also serves the frontend
│   ├── requirements.txt
│   └── services/
│       ├── parser.py         # PDF / DOCX / TXT  ->  text
│       ├── scorer.py         # hybrid scoring + ranking
│       └── explainer.py      # Claude rationale (+ offline fallback)
├── frontend/
│   └── index.html            # single-page UI (drag-drop, results)
├── sample_data/              # a JD + 3 example resumes to test with
└── README.md
```

`main.py`	Defines the API, handles file uploads, orchestrates parse → score → explain, and mounts the frontend.
`services/parser.py`	Extracts plain text from PDF (pdfplumber), DOCX (python-docx), and TXT files.
`services/scorer.py`	The core engine: embeddings, skill matching, fuzzy overlap, weighting, and ranking. **Weights live at the top of this file.**
`services/explainer.py`	Generates the per-candidate rationale via Claude, with a no-key fallback.
`frontend/index.html`	Self-contained UI — no build step, no framework. Drag-drop upload and ranked result cards.

05 Installation & Running

Prerequisite: Python 3.10 or newer installed and on your PATH. Check with `python --version`.

> **Run the commands one line at a time**, not all pasted together. The virtual-environment activation command differs between Windows and macOS/Linux — use the right one for your system.

Windows (PowerShell)

```
PS> cd resume-screener\backend
PS> python -m venv venv
PS> venv\Scripts\activate
PS> pip install -r requirements.txt
PS> uvicorn main:app --reload --port 8000
```

If activation is blocked by an execution-policy error, run this once, then activate again:

```
PS> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

macOS / Linux

```
$ cd resume-screener/backend
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install -r requirements.txt
$ uvicorn main:app --reload --port 8000
```

> When activation succeeds, your prompt shows `(venv)` at the start. Then open **http://localhost:8000** in your browser — backend and UI are on that one URL.

First run note: the embedding model (`all-MiniLM-L6-v2`, ~90 MB) downloads once on first use and is cached afterward, so subsequent launches are fast. An internet connection is needed only for that first download.

Quick test

Paste the contents of `sample_data/job_description.txt` into the role box and upload the three `sample_data/candidate_*.txt` files. Candidate A should rank #1.

06 Using the Application

1. **Paste the job description** into the role box on the left.
2. **Add must-have skills** (optional), comma-separated, e.g. `python, fastapi, faiss, docker`. If left blank, skills are auto-extracted from the JD.
3. **Set "LLM rationale for top"** — how many of the highest-ranked candidates receive a written rationale (set to 0 to skip).
4. **Upload resumes** by dragging files onto the drop zone or clicking to browse. PDF, DOCX, and TXT are supported; multiple files at once.
5. **Click "Screen candidates."** Results appear on the right, ranked best-first.

Reading a result card

- **Rank & overall score** — the headline 0–100 composite.
- **Three metrics** — semantic fit, skill coverage, text overlap, shown individually.
- **Rationale** — the plain-language explanation (top-N only).
- **Skill chips** — green = matched, orange = missing from the resume.

07 API Reference

The frontend talks to two endpoints. You can also call them directly (e.g. from Postman or another service).

GET /api/health

Liveness check. Returns `{ "status": "ok" }`.

POST /api/screen

Accepts a `multipart/form-data` request and returns the ranked results.

Field	Type	Required	Description
`job_description`	text	Yes	The full job description.
`skills`	text	No	Comma-separated must-have skills. Blank = auto-extract.
`explain_top`	int	No	How many top candidates get an LLM rationale (default 3).
`resumes`	file[]	Yes	One or more resume files (PDF / DOCX / TXT).

Response shape

```
{
  "count": 3,
  "errors": [],
  "results": [
    {
      "rank": 1,
      "name": "candidate_a.pdf",
      "score": 96.4,
      "semantic": 71.2,
      "skill_coverage": 100.0,
      "fuzzy": 48.0,
      "matched_skills": ["python", "fastapi", "faiss"],
      "missing_skills": [],
      "rationale": "Strong end-to-end match ..."
    }
  ]
}
```

Full resume text is intentionally stripped from the response. The `errors` array lists any files that could not be read, so a single bad upload never fails the whole batch.

08 Configuration & Tuning

Adjusting the weights

The three weights are constants at the top of `backend/services/scorer.py`:

```
W_SEMANTIC = 0.55   # conceptual match
W_SKILL    = 0.35   # required-skill coverage
W_FUZZY    = 0.10   # lexical overlap tiebreaker
```

Raise `W_SKILL` for roles with hard, non-negotiable skill requirements; raise `W_SEMANTIC` for roles where transferable experience matters more. Keep the three summing to 1.0.

Skill-match strictness

In `_skill_match()`, the fuzzy threshold `partial_ratio ≥ 90` controls how forgiving skill matching is. Lower it (e.g. 80) to catch more spelling variants at the cost of some false positives.

Enabling real LLM rationales

Set an Anthropic API key before launching. Without it, the deterministic fallback is used.

```
# Windows
PS> set ANTHROPIC_API_KEY=sk-ant-...
# macOS / Linux
$ export ANTHROPIC_API_KEY=sk-ant-...
```

Swapping the embedding model

Change the model name in `_model()` in `scorer.py`. A larger model (e.g. `all-mpnet-base-v2`) improves semantic quality at the cost of speed.

09 Troubleshooting

Symptom	Cause & fix
`Cannot find path ... \backend`	You're one level too high, or the zip extracted nested. Run `dir`; if you see another `resume-screener` folder, `cd` into it first.
`'source' is not recognized`	`source` is macOS/Linux only. On Windows use `venv\Scripts\activate`.
Activation blocked / "running scripts is disabled"	Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again.
First request hangs, then mentions huggingface.co	The embedding model is downloading (~90 MB). Needs internet on first run only; wait for it to finish, then it's cached.
`uvicorn: command not found`	The venv isn't active (no `(venv)` in the prompt) or deps didn't install. Re-activate and re-run `pip install -r requirements.txt`.
Port 8000 already in use	Launch on another port: `uvicorn main:app --reload --port 8001`.
A resume shows "no extractable text"	It's likely a scanned/image-only PDF. OCR isn't included; convert it to a text PDF or DOCX first.

10 Limitations & Future Work

Current limitations

- **No OCR** — scanned image-only PDFs yield no text. Pair with an OCR step to handle them.
- **Brute-force similarity** — fine for a few hundred resumes; for tens of thousands, index embeddings in FAISS.
- **Stateless** — screening runs aren't saved. Add a database to persist and audit results.
- **Skills auto-extraction is heuristic** — providing the explicit skills list always gives better results.

Roadmap ideas

- **Bias mitigation:** strip name, gender, and age signals before scoring for fairer screening and compliance.
- **FAISS backend:** swap brute-force cosine for an ANN index to scale to large candidate pools.
- **Persistence & history:** store runs, allow re-ranking and side-by-side comparison.
- **Export:** downloadable CSV / PDF shortlist for sharing with hiring managers.
- **Configurable rubrics:** per-role weight presets saved in the UI.

> **In short:** Sift is a transparent, hybrid first-pass screener built to assist recruiters — fast to run, easy to tune, and honest about why each candidate ranks where they do.